

BERLINER HOCHSCHULE FÜR TECHNIK
MOBILE GEOANWENDUNGEN, Sommersemester 2026

Kaskade

Kurzbeschreibung

Carl Janek Markert (Matrikelnummer 105906)

Paul Tschätsch (Matrikelnummer 107359)

22. September 2026

Worum es geht

Kaskade ist eine native iOS-App, die vier Ortungsverfahren – GNSS, Ultrabreitband (UWB), Bluetooth-RSSI (BLE) und LiDAR – nach demselben Protokoll gegen einen unabhängigen Referenzwert vermisst: Sollwert, Aufzeichnung, CSV-Export, Auswertung. Als Anwendungsfall, der diese Ortungskaskade motiviert, klassifiziert die App zusätzlich den Transportmodus einer Fahrt über ein selbst trainiertes Modell (Fuß, Rad, Auto, Zug) – eine Unterscheidung, die Apples Systemklassifikator `CMMotionActivity` strukturell nicht leisten kann.

Vier Messstudien mit echten Felddaten: GNSS-Genauigkeit an drei ALKIS-Referenzpunkten (M1), Transportmodus-Klassifikation gegen die Systembaseline (M2), UWB gegen BLE im Nahbereich (M3) und LiDAR gegen ein Maßband (M4). M1, M3 und M4 belegen die vier Ortungsverfahren, M2 prüft den Anwendungsfall. Details, Zahlen und Grafiken stehen im vollständigen Abgabedokument `abgabe.pdf`.

Ergebnisse auf einen Blick

Studie	Ergebnis
M1 GNSS	CEP95 an drei Referenzpunkten: 5,1 m (abgeschattet), 14,0 m (offener Himmel), 32,3 m (Straßenschlucht, bimodal: CEP50 nur 0,38 m)
M3 UWB/BLE	UWB 0,2 bis 0,4 m mittlerer Fehler bei fünf von sieben Distanzen, Reichweitengrenze bei 8 m; BLE-RSSI bis 4,7 m Fehler bei 8 m
M4 LiDAR	1 bis 29 mm Fehler von 0,3 bis 5 m; 0,55 m bei 17 m, außerhalb der Spezifikation
M2 Transportmodus	sechs Segmente: Macro-F1 0,54 (eigenes Modell) gegen 0,62 (<code>CMMotionActivity</code>); die Baseline kann <i>train</i> nicht vergeben, das Modell erkennt es, verfehlt dafür die Radfahrt

Technologien

Bereich	Technologie
Positionierung	CoreLocation (GNSS), ausgewertet gegen ALKIS-Referenzpunkte
Nahbereichsortung	NearbyInteraction (UWB), CoreBluetooth/CLBeaconRegion (BLE-RSSI)
Nahfeldmessung	ARKit <code>sceneDepth</code> (LiDAR)
Klassifikation	scikit-learn Random Forest, exportiert über coremltools nach Core ML
Karten	MapLibre GL (iOS-App und Native), MapLibre GL JS (Standalone-Viewer)
Speicherung	SQLite über GRDB, CSV-/GeoJSON-Export
Plattform	Natives SwiftUI, iOS 17+

Aufbau des Repositories

ios/ SwiftUI-App (Xcode-Projekt aus `project.yml` über XcodeGen erzeugt)

analysis/ Python-Training und -Auswertung, `run_all.py` erzeugt alle Abbildungen/Tabellen neu

data/ eingecheckte Rohmessreihen, Tracks und Referenzpunkte

viewer/ eigenständiger Betrachter für exportierte Feldtest-Tracks (MapLibre GL JS)

docs/abgabe/ vollständiges Abgabedokument

Screenshots

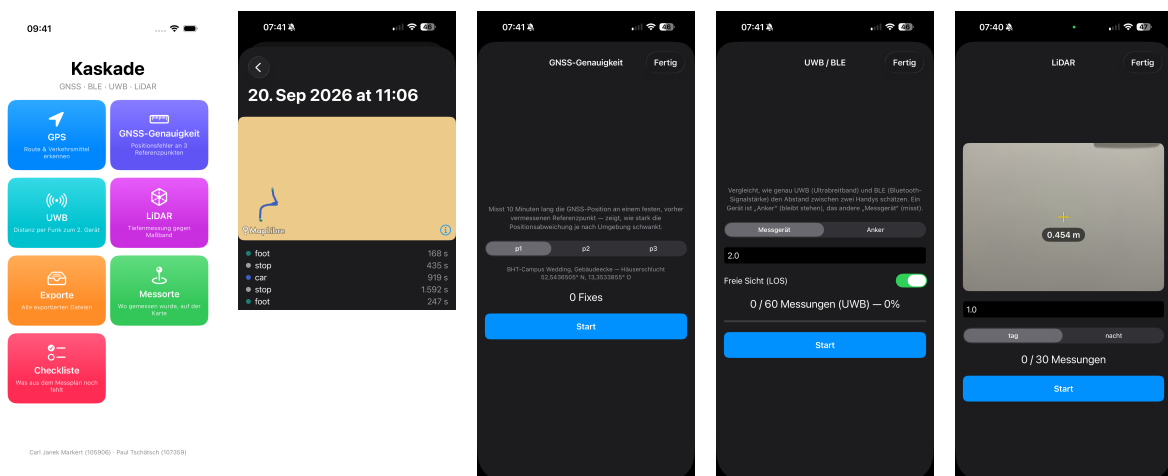


Abbildung 1: Von links: Startbildschirm mit den sieben Werkzeugen; Trip vom 20.09. mit Karte und Segmentierung in Fußwege, Stopps und Fahrt (Handlabels `foot` und `car`); M1, GNSS-Genauigkeit am Referenzpunkt p1 (BHT-Campus Wedding, Häuserschlucht), zehn Minuten Messdauer; M3, UWB und BLE-RSSI gegen eine Solldistanz mit Anker und Messgerät, 60 Messungen je Durchgang; M4, LiDAR-Tiefe im Fadenkreuz gegen ein Maßband, 30 Messungen je Durchgang, getrennt nach Tag und Nacht.

Arbeitsteilung

Carl Janek Markert: Transportmodus-Klassifikation, Segmentierung, Auswertung M1/M2. Paul Tschätsch: UWB-, BLE- und LiDAR-Messung, Auswertung M3/M4. Details und die eingefrorene Schnittstelle zwischen beiden Arbeitspaketen stehen im Abgabedokument.